

```
from google.colab import drive
drive.mount('/content/drive')
```

↗ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout, BatchNormalization
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve
```

```
# Load feature and target data
features = pd.read_csv(r"/content/drive/MyDrive/Colab Notebooks/dl/dataset/ECG PCG feature(128+128=256).csv", header=None, skiprows=[0]) # A
targets = pd.read_csv(r"/content/drive/MyDrive/Colab Notebooks/dl/dataset/Reference_aug.csv", header=None)
```

```
print(features.shape)
print(targets.shape)
```

```
# Extract target column
y = targets.iloc[:, -1] # Selecting the last column
X = features # All feature columns
```

```
# Using .apply() method
y = y.apply(lambda x: max(x, 0))
```

↗ (1975, 256)
(1975, 2)

```
# Split data into training and testing sets
Xtrain, Xtest, Ytrain, Ytest = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Standardize features
scaler = StandardScaler()
Xtrain_scaled = scaler.fit_transform(Xtrain)
Xtest_scaled = scaler.transform(Xtest)
```

```
# Reshape data for Conv1D input
Xtrain_resaped = Xtrain_scaled.reshape((Xtrain_scaled.shape[0], Xtrain_scaled.shape[1], 1))
Xtest_resaped = Xtest_scaled.reshape((Xtest_scaled.shape[0], Xtest_scaled.shape[1], 1))
```

✓ CNN

```
model = Sequential([
    Conv1D(filters=8, kernel_size=3, activation='relu', input_shape=(256, 1)),
    BatchNormalization(),
    Dropout(0.5),
    Conv1D(filters=16, kernel_size=3, activation='relu'),
    BatchNormalization(),
    Dropout(0.5),
    Conv1D(filters=32, kernel_size=3, activation='relu'),
    BatchNormalization(),
    Dropout(0.5),
    Conv1D(filters=16, kernel_size=3, activation='relu'),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(128, activation='relu'),
    Dense(128, activation='relu'),
    Dense(128, activation='relu'),
    Dense(1, activation='sigmoid') # 2 output classes for binary classification
])
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

↗ /usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input\_shape` to `input\_shape` in the constructor of `Conv1D` or `Conv2D` layers. It is deprecated and will be removed in a future version. Use the `input\_shape` argument in the `compile` method instead. (activity_regularizer=activity_regularizer, **kwargs)

```
# Train the model
history = model.fit(Xtrain_resaped, Ytrain, epochs=100, batch_size=32, validation_split=0.2)
```

 Show hidden output

```
# Evaluate the model on the test set
loss, accuracy = model.evaluate(Xtest_resaped, Ytest)
print(f"Test Accuracy: {accuracy:.2%}")
print(f"Test Loss: {loss:.4f}")
```

 13/13  0s 13ms/step - accuracy: 0.7122 - loss: 1.3846
Test Accuracy: 74.43%
Test Loss: 1.2731

```
Ypred = model.predict(Xtest_resaped)
Ypred_classes = (Ypred > 0.5).astype(int)
Ypred_classes_flat = Ypred_classes.flatten()
```

```
# Print classification report (precision, recall, F1-score)
print("\nClassification Report:")
print(classification_report(Ytest, Ypred_classes_flat)) # Use Ytest directly
```

 13/13  0s 23ms/step

```
Classification Report:
              precision    recall  f1-score   support

     0       0.71      0.79      0.75        190
     1       0.78      0.70      0.74        205

 accuracy          0.74          395
 macro avg         0.75          395
 weighted avg      0.75          395
```

```
# Plot accuracy
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

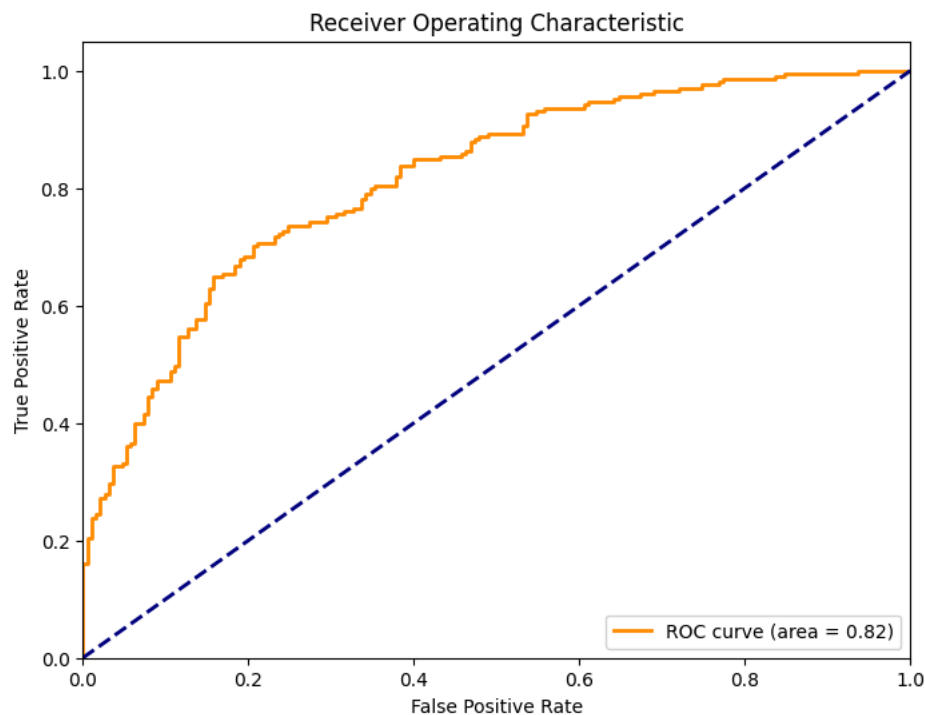


```
roc_auc = roc_auc_score(Ytest, Ypred[:, 0])
print(f"\nROC AUC: {roc_auc:.4f}")
```

```
# Plot ROC curve
# Use Ytest for true labels and Ypred[:, 0] for predicted probabilities of the positive class
fpr, tpr, thresholds = roc_curve(Ytest, Ypred[:, 0])
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```



ROC AUC: 0.8155



```
cm = confusion_matrix(Ytest, Ypred_classes_t1at)
plt.figure(figsize=(8, 6))
class_labels = ['Class 0', 'Class 1'] # Or provide your actual class names if available

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_labels,
            yticklabels=class_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

